

Courier Tracking System

INTRODUCTION:

The Courier Tracking System is a versatile and user-friendly web-based application designed to streamline the process of booking, managing, and tracking courier shipments. Suitable for individuals, businesses, and logistics providers, it offers a wide range of features including shipment booking, real-time tracking, delivery scheduling, and status notifications—all accessible through an intuitive interface. Users can easily enter shipment details, select delivery options such as express, standard, or economy, and generate a unique tracking ID to monitor their parcels at every stage of the delivery process.

The system provides a customizable and responsive user experience, allowing users to manage their shipments efficiently while ensuring data accuracy and transparency. Robust security features such as user authentication, encrypted data handling, and secure API integration help protect sensitive information and maintain system reliability. Additionally, the platform supports seamless integration with third-party services like payment gateways and notification systems to automate transactions and updates.

With built-in tracking visualization and reporting capabilities, users and administrators can monitor shipment progress, analyze delivery performance, and maintain detailed records for future reference. Its flexibility, simplicity, and powerful functionality make the Courier Tracking System an ideal solution for modern delivery management, helping organizations improve operational efficiency and enhance customer satisfaction.

SCENARIO Based Case Study:

Meet Rahul, a small business owner who manages an online store and regularly ships products to customers across different cities. Rahul has a busy schedule and values efficiency and organization in his work. He is responsible for managing orders, ensuring timely deliveries, and keeping customers informed about their shipment status. However, handling courier operations manually becomes difficult, as it requires constant tracking, communication, and record management.

Rahul discovers the Courier Tracking System, a comprehensive web application designed to simplify and streamline courier and delivery operations. Impressed by its features, Rahul decides to explore how the system can improve his workflow and enhance his business efficiency.

User Registration and Authentication: Rahul creates an account on the Courier Tracking System by entering his personal and business details. He logs in securely using his

credentials, ensuring that his shipment data and customer information remain safe and protected.

Shipment Booking Management: Rahul uses the system to book shipments by entering sender and receiver details, package information, and selecting the preferred delivery option such as express, 2-day delivery, or standard delivery. This allows him to quickly process orders without manual paperwork.

Real-Time Tracking System: Once a shipment is booked, the system generates a unique tracking ID. Rahul uses this ID to monitor the status of his parcels in real time. He can see updates such as “Order Placed,” “Dispatched,” “In Transit,” “Out for Delivery,” and “Delivered,” which helps him stay informed at every stage.

Customer Communication Management: The system enables Rahul to share tracking details with his customers, allowing them to check their order status independently. This reduces the need for constant communication and improves customer satisfaction.

Order History and Reporting: Rahul can view all his past and current shipments in one place. The system maintains detailed records, allowing him to analyze delivery performance and manage orders efficiently.

Admin and Delivery Operations: The Courier Tracking System allows administrators to update shipment statuses, assign delivery agents, and manage logistics. Delivery agents update the delivery progress, ensuring accurate and timely information.

Support and Assistance: The system provides help resources such as FAQs and support options, helping Rahul resolve any issues quickly and continue his operations smoothly.

Rahul’s Experience: With the help of the Courier Tracking System, Rahul is now able to manage his delivery operations more efficiently and effectively. He spends less time tracking shipments manually and more time focusing on growing his business. The system improves transparency, reduces errors, and enhances customer trust, making it an essential tool in Rahul’s daily work.

SYSTEM REQUIREMENTS:

Software Requirements:

- To ensure smooth development, deployment, and usage of the Courier Tracking System web application, certain system prerequisites must be met. These requirements are categorized into software and hardware specifications, all of which contribute to building a robust, scalable, and efficient courier management platform.

1. Software Requirements

- These are the essential tools and platforms required to develop, test, and run the application efficiently.
- Operating System: Windows 10/11, macOS, or Linux — Supports cross-platform development, testing, and deployment of the application.
- Node.js (v16 or above): Provides the runtime environment for executing JavaScript on the server side and is responsible for handling backend logic, API requests, and server operations.
- npm (v8 or above): A package manager used to install and manage dependencies required for both frontend and backend development.
- React.js: A powerful JavaScript library used to build dynamic, responsive, and interactive user interfaces for the courier tracking system.
- Browser: Google Chrome / Mozilla Firefox (latest version) — Used for rendering the application and testing user interface behavior in real time.
- Express.js: A lightweight and flexible web framework for Node.js used to build RESTful APIs and handle server-side routing.
- MongoDB: A NoSQL database used to store user data, shipment details, tracking information, and system records in a structured and efficient manner.
- Postman: A development tool used to test APIs, send requests, and verify backend functionality during development.
- Visual Studio Code: A widely used code editor that provides features like syntax highlighting, debugging, Git integration, and terminal support for efficient coding.

2. Hardware Requirements

- This section describes the minimum and recommended hardware specifications needed to support the development and smooth operation of the Courier Tracking System.
- Processor: Intel Core i5 (8th Generation or above) / AMD Ryzen 5 or better — Ensures efficient processing, faster compilation, and smooth multitasking during development and execution.
- RAM: Minimum 8 GB (16 GB recommended) — Required for running development servers, code editors, browsers, and database systems

simultaneously without performance issues.

- Storage: At least 1 GB of free disk space — Needed for installing dependencies, setting up the database, and storing project files.
- Display: Resolution of 1366x768 or higher — Recommended for better visualization of the user interface and improved coding experience

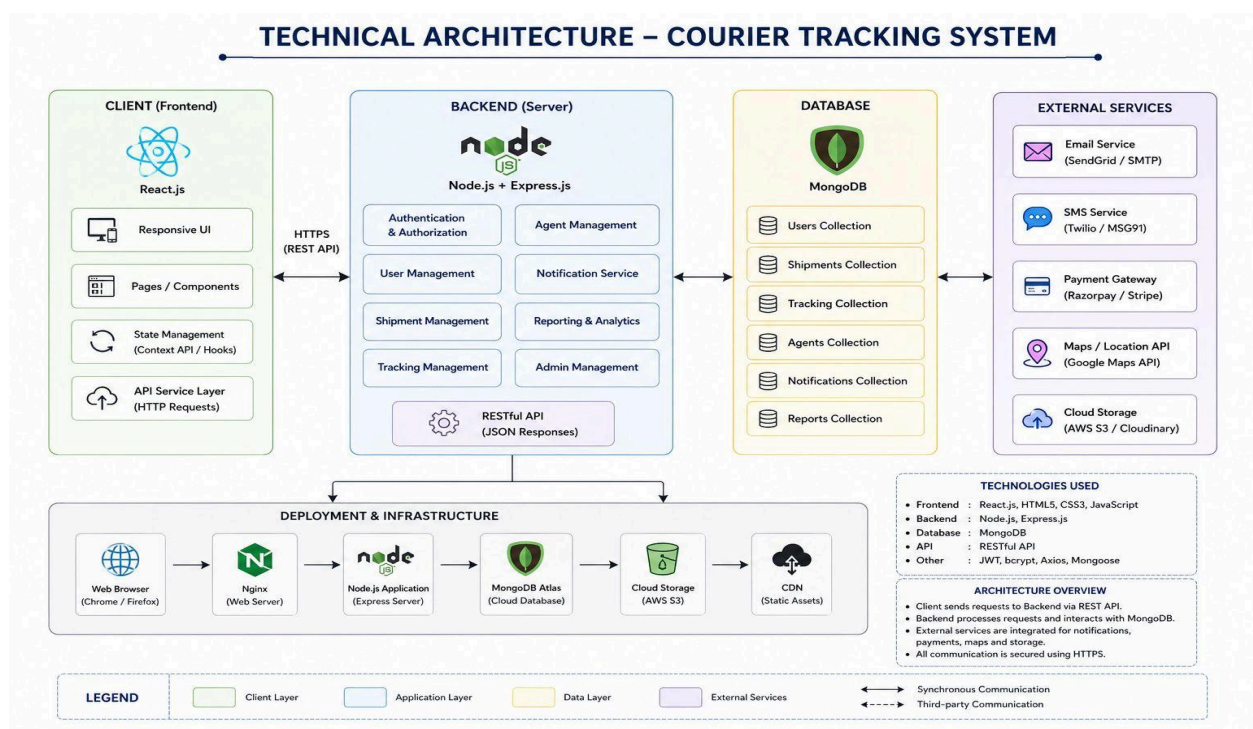
PROJECT ARCHITECTURE:

The Courier Tracking System follows a structured architecture that separates the application into different layers for better organization, scalability, and performance. The system is mainly divided into three main components which are the frontend, backend, and database.

The frontend is responsible for user interaction and is developed using technologies like React.js. It provides interfaces for users to register, log in, book shipments, and track parcels easily. The backend is developed using Node.js and Express.js, which handle all the business logic, API requests, and data processing. The database is implemented using MongoDB, which stores all user information, shipment details, and tracking records securely.

This layered architecture ensures smooth communication between different components and allows the system to be easily maintained, updated, and scaled in the future.

TECHNICAL ARCHITECTURE:



In this architecture, the flow begins with a user (represented by the "User" node) interacting with the Courier Tracking System frontend (represented by the "Frontend" node). The user performs actions such as signing up, logging in, booking a shipment, or tracking a parcel through the interface.

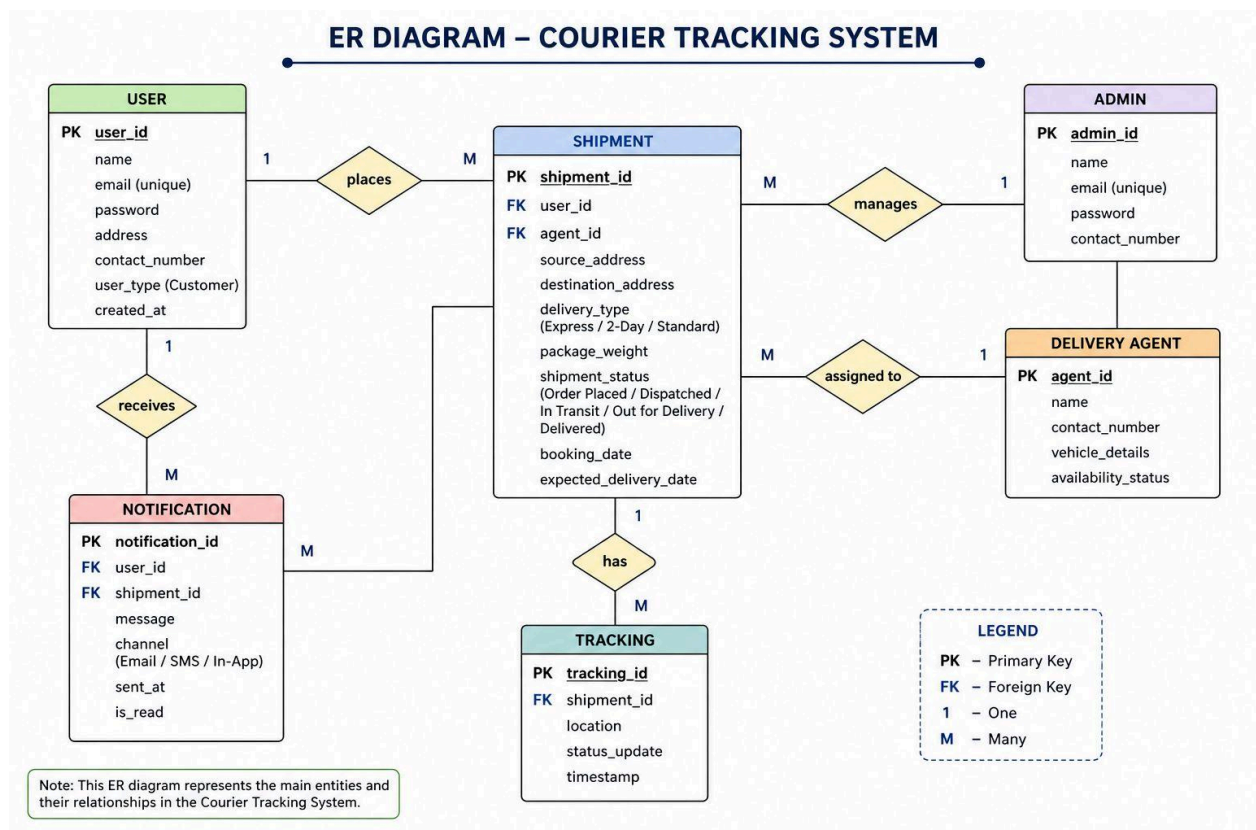
The frontend then sends a request to the backend server (represented by the "Backend" node). The backend processes the request and validates the input data. It performs operations such as creating a shipment, updating tracking status, or retrieving shipment details.

The backend communicates with a database (represented by the "Database" node) to store and retrieve information. The database securely stores user data, shipment records, tracking updates, and other system details.

Additionally, the backend may use a file storage system (represented by the "File Storage" node) to store files such as shipment receipts, invoices, or proof of delivery images uploaded by users or delivery agents.

Once the backend has processed the request, it sends a response back to the frontend. The frontend then displays the output to the user, such as booking confirmation or tracking updates.

ER DIAGRAM:



The Entity-Relationship (ER) diagram for the Courier Tracking System represents the various entities and their relationships within the system. It helps in understanding how

data is organized and connected.

1. Entities:

User: Represents customers who use the system to send and track parcels, with attributes such as user ID, name, email, password, address, and contact number.

Shipment: Represents courier orders placed by users, with attributes such as shipment ID, source address, destination address, delivery type, and status.

Tracking: Represents tracking details of shipments, with attributes such as tracking ID, location, time, and status updates.

Admin: Represents system administrators who manage the platform, with attributes such as admin ID, name, and login details.

Delivery Agent: Represents delivery personnel responsible for transporting parcels, with attributes such as agent ID, name, and contact information.

2. Relationships:

User - Shipment: Represents the relationship where a user places shipments.

Shipment - Tracking: Represents the relationship where each shipment has tracking updates.

Admin - Shipment: Represents the relationship where the admin manages shipments.

Delivery Agent - Shipment: Represents the relationship where a delivery agent is assigned to deliver shipments.

3. Attributes:

Each entity has its own set of attributes that describe its properties. For example, the Shipment entity includes shipment ID, source, destination, and status.

4. Primary Keys:

User: user ID

Shipment: shipment ID

Tracking: tracking ID

Admin: admin ID

Delivery Agent: agent ID

5. Foreign Keys:

Shipment contains user ID to link shipments with users.

Tracking contains shipment ID to link tracking updates with shipments.

Shipment may contain agent ID to assign delivery agents.

Overall, the ER diagram provides a clear structure of the database and shows how different entities are connected and how data flows within the Courier Tracking System

Key Features:

The key features of a Courier Tracking System can vary depending on its functionality and requirements. However, some common features include:

- **User Registration and Profiles:** Allow users to create accounts, log in securely, and manage personal details such as name, contact, and address.
- **Shipment Booking Management:** Allow users to enter sender and receiver details, choose delivery options like express, standard, or economy, and book courier services

easily.

- **Real-Time Tracking:** Provide a unique tracking ID to monitor shipment status with updates like dispatched, in transit, out for delivery, and delivered.
- **Payment Integration:** Enable secure online payments using methods like cards, UPI, or digital wallets.
- **Notifications and Updates:** Send real-time alerts to inform users about shipment progress.
- **Order History:** Maintain records of past and current shipments for easy reference.
- **Admin Dashboard:** Allow admins to manage users, update shipment status, and monitor system operations.
- **Delivery Agent Management:** Assign delivery agents to shipments and allow them to update delivery status.
- **Support and Assistance:** Provide help through FAQs or customer support to resolve issues.

These are general features of the Courier Tracking System and can be customized based on the Requirements.

ROLES AND RESPONSIBILITIES:

User:

Registration and Account Management: Users are responsible for creating an account, providing accurate personal information, and managing their account details, including profile updates and password security.

Shipment Booking: Users can enter sender and receiver details, choose delivery options such as express, standard, or economy, and book courier services based on their requirements. They are responsible for providing correct shipment information.

Payment: Users are responsible for making payments for the booked shipments through available payment methods and should ensure the accuracy and security of their payment details.

Tracking and Monitoring: Users can track their shipments using a tracking ID and stay updated with delivery status at every stage.

Communication: Users may communicate with customer support for inquiries, updates, or issues related to their shipments and should use the provided communication channels properly.

Feedback and Reviews: Users have the responsibility to provide honest feedback on the delivery services, helping improve the system and assisting other users

Admins:

System Management: The admin is responsible for overall management of the Courier Tracking System, including monitoring system performance, ensuring data security, and maintaining system settings.

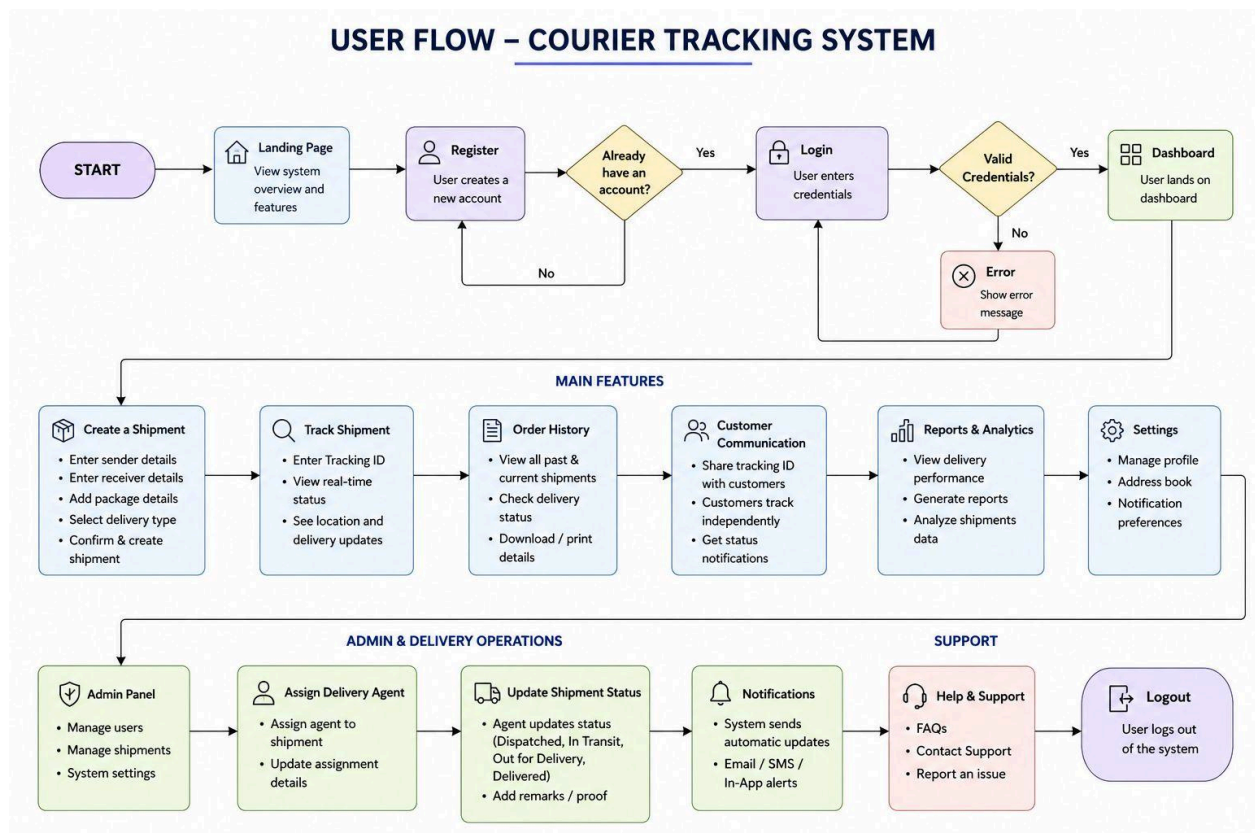
User Management: The admin manages user accounts, including registration, verification, updates, and handling account-related issues such as suspension or deletion when necessary.

Shipment Management: The admin monitors all shipments, updates delivery status, and ensures smooth operation of courier services.

Delivery Agent Management: The admin assigns delivery agents to shipments, tracks their performance, and ensures timely delivery.

Data Management: The admin maintains records of users, shipments, and tracking details, ensuring accuracy and proper database management

User Flow:



Customer Flow

Registration & Account – Create an account and manage personal details securely.

Shipment Booking – Enter sender and receiver details, select delivery options, and book courier services.

Payment – Make secure payments using available methods.

Tracking – Monitor shipment status in real time using a tracking ID.

Communication – Contact support for queries, updates, or issues related to shipments.

Feedback – Provide reviews and feedback to improve services.

Admin Flow

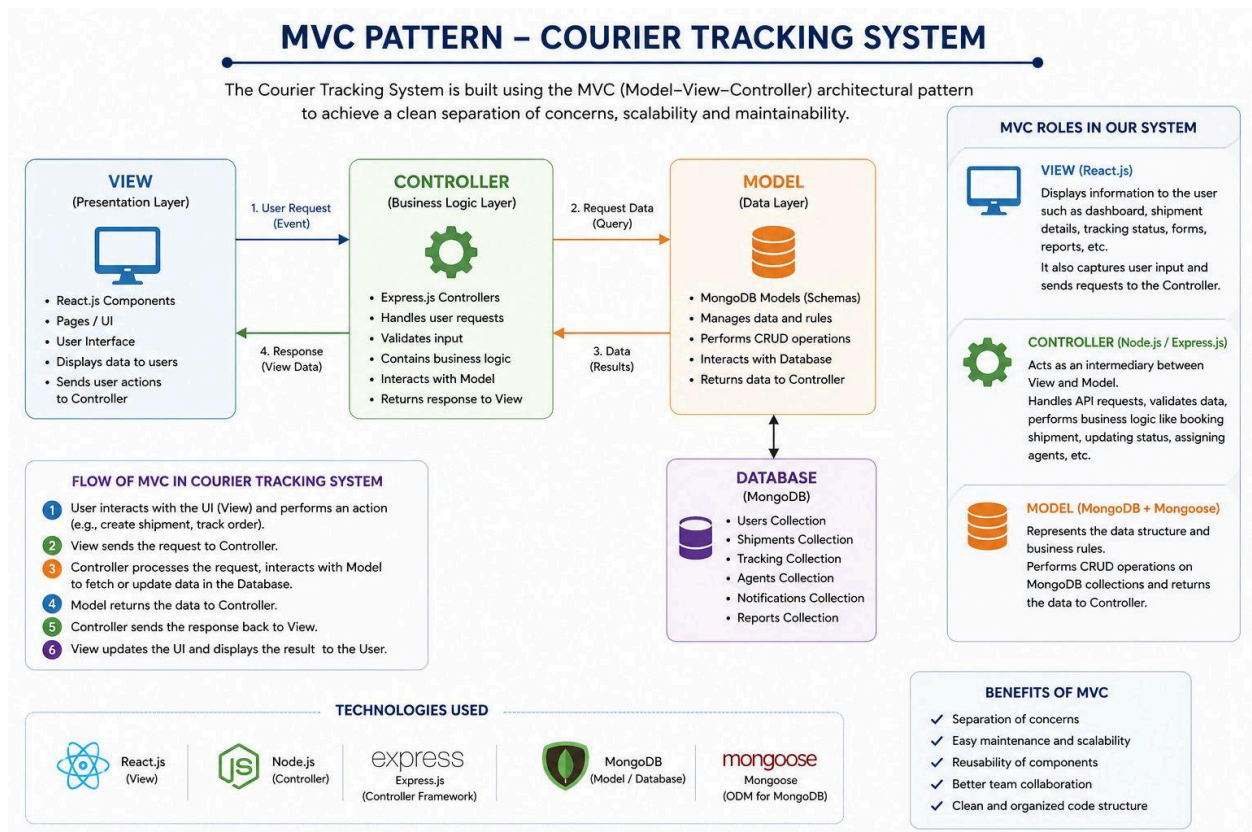
System Management – Monitor system performance, settings, and data security.

User Management – Verify, manage, or control user accounts.

Shipment Management – Oversee all shipments and update delivery status.

Delivery Agent Management – Assign and manage delivery agents for shipments

MVC PATTERN:



The Courier Tracking System follows the Model-View-Controller (MVC) architectural pattern, which is a software design approach that separates the application into three interconnected layers. This separation helps in improving modularity, maintainability, and scalability of the system.

Model Layer (Data Layer)

The Model layer is responsible for handling all data-related operations. It defines the structure of the data and manages interactions with the database. In this project, models such as User, Shipment, and Tracking are created using Mongoose to store and manage data in MongoDB efficiently.

Controller Layer

The Controller layer acts as a bridge between the View and the Model. It receives requests from the frontend, processes the input data, performs validation, and calls the appropriate model functions. It then sends the processed response back to the user. For example, it handles operations like booking shipments, updating tracking status, and retrieving shipment details.

View Layer (Routing Layer)

The View layer represents the user interface and routing system. It defines how users interact with the application through pages and API endpoints. It handles HTTP requests

such as GET, POST, PUT, and DELETE, and connects them to the respective controller functions to display the required output.

Advantages of Using MVC in This Project

Separation of Concerns: Each layer has a specific role, making the system easier to understand and maintain.

Scalability: New features like advanced tracking or payment modules can be added easily.

Reusability: Code written in models and controllers can be reused across different parts of the system.

Testing: Each layer can be tested independently, improving reliability.

Collaboration-Friendly: Multiple developers can work on different layers without conflicts.

PROJECT SETUP AND CONFIGURATION:

Creating project folder

Create a new folder with your project name. Inside that folder create two folders named Client and Server. Now open the main folder in Visual Studio Code.

Client setup (installing react app)

Open the Client folder in the terminal of VS Code.

`npm create vite@latest`

Select React framework → React

Select variant → JavaScript

Navigate to client folder: `cd client`

Install packages: `npm install`

Start server: `npm run dev`

Server setup (npm init)

Open Server folder in terminal of VS Code.

`npm init -y`

Create file: `server.js`

Create folders: `models`, `controllers`, `route`.

BACKEND DEVELOPMENT

Backend Reference Video:

https://drive.google.com/file/d/1iK3tdUiKPUU4wHimqsDsk9eU6VTBM0La/view?usp=drive_sdk

Backend Structure:

Backend Folder Structure

```
txt

backend/
  src/
    controllers/
      adminController.js
      userController.js
      agentController.js
    db/
      config.js
    middleware/
      authMiddleware.js
      upload.js
    models/
      Admin/
        Admins.js
      User/
        Users.js
      Shipment/
        Shipments.js
        Tracking.js
      DeliveryAgent/
        Agents.js
    routes/
      adminRoutes.js
      userRoutes.js
      agentRoutes.js
    app.js
    server.js
```

backend/src/db/config.js

```
js

import mongoose from "mongoose";

const connectDB = async () => {
  try {
    const connection = await mongoose.connect(process.env.MONGO_URI);
    console.log(`MongoDB connected: ${connection.connection.host}`);
  } catch (error) {
    console.error(`Database connection failed: ${error.message}`);
    process.exit(1);
  }
};

export default connectDB;
```

backend/src/models/Admin/Admins.js

```
js

import bcrypt from "bcryptjs";
import mongoose from "mongoose";

const adminSchema = new mongoose.Schema(
  {
    name: { type: String, required: true, trim: true },
    email: { type: String, required: true, unique: true, lowercase: true },
    password: { type: String, required: true, minlength: 8, select: false },
    role: { type: String, default: "admin" }
  },
  { timestamps: true }
);

adminSchema.pre("save", async function save(next) {
  if (!this.isModified("password")) return next();
  this.password = await bcrypt.hash(this.password, 10);
  return next();
});

adminSchema.methods.matchPassword = function matchPassword(password) {
  return bcrypt.compare(password, this.password);
};

export default mongoose.model("Admin", adminSchema);
```

backend/src/models/User/Users.js

```
js

import bcrypt from "bcryptjs";
import mongoose from "mongoose";

const userSchema = new mongoose.Schema(
  {
    name: { type: String, required: true, trim: true },
    email: { type: String, required: true, unique: true, lowercase: true },
    phone: { type: String, trim: true },
    address: { type: String, trim: true },
    password: { type: String, required: true, minlength: 8, select: false },
    isActive: { type: Boolean, default: true }
  },
  { timestamps: true }
);

userSchema.pre("save", async function save(next) {
  if (!this.isModified("password")) return next();
  this.password = await bcrypt.hash(this.password, 10);
  return next();
});

userSchema.methods.matchPassword = function matchPassword(password) {
  return bcrypt.compare(password, this.password);
};

export default mongoose.model("User", userSchema);
```

backend/src/models/Shipment/Shipments.js

```
js

import mongoose from "mongoose";

const addressSchema = new mongoose.Schema(
  {
    name: { type: String, required: true },
    phone: { type: String, required: true },
    email: { type: String },
    address: { type: String, required: true },
    city: { type: String },
    pincode: { type: String }
  },
  { _id: false }
);

const shipmentSchema = new mongoose.Schema(
  {
    trackingId: { type: String, required: true, unique: true },
    user: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
    agent: { type: mongoose.Schema.Types.ObjectId, ref: "Agent" },
    sender: { type: addressSchema, required: true },
    receiver: { type: addressSchema, required: true },
    packageWeight: { type: Number, required: true },
    deliveryType: { type: String, enum: ["Standard", "Express", "Economy"], default: "Standard" },
    status: {
      type: String,
      enum: ["Booked", "Picked Up", "In Transit", "Out For Delivery", "Delivered", "Cancelled"],
      default: "Booked"
    },
    deliveryProof: { type: String },
    estimatedDelivery: { type: Date }
  },
  { timestamps: true }
);

export default mongoose.model("Shipment", shipmentSchema);
```

backend/src/models/Shipment/Tracking.js

```
js

import mongoose from "mongoose";

const trackingSchema = new mongoose.Schema(
  {
    shipment: { type: mongoose.Schema.Types.ObjectId, ref: "Shipment", required: true },
    status: { type: String, required: true },
    location: { type: String, required: true },
    message: { type: String, default: "Shipment update recorded" },
    updatedBy: { type: String, enum: ["admin", "agent", "system"], default: "system" }
  },
  { timestamps: true }
);

export default mongoose.model("Tracking", trackingSchema);
```

backend/src/models/DeliveryAgent/Agents.js

```
js

import bcrypt from "bcryptjs";
import mongoose from "mongoose";

const agentSchema = new mongoose.Schema(
  {
    name: { type: String, required: true, trim: true },
    email: { type: String, required: true, unique: true, lowercase: true },
    phone: { type: String, required: true },
    password: { type: String, required: true, minlength: 8, select: false },
    area: { type: String, required: true },
    assignedShipments: [{ type: mongoose.Schema.Types.ObjectId, ref: "Shipment" }],
    isAvailable: { type: Boolean, default: true }
  },
  { timestamps: true }
);

agentSchema.pre("save", async function save(next) {
  if (!this.isModified("password")) return next();
  this.password = await bcrypt.hash(this.password, 10);
  return next();
});

agentSchema.methods.matchPassword = function matchPassword(password) {
  return bcrypt.compare(password, this.password);
};

export default mongoose.model("Agent", agentSchema);
```

backend/src/middleware/authMiddleware.js

```
js

import jwt from "jsonwebtoken";
import User from "../models/User/Users.js";
import Admin from "../models/Admin/Admins.js";
import Agent from "../models/DeliveryAgent/Agents.js";

export const protect = async (req, res, next) => {
  try {
    const token = req.headers.authorization?.startsWith("Bearer")
      ? req.headers.authorization.split(" ")[1]
      : null;

    if (!token) return res.status(401).json({ message: "Not authorized" });

    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    const Model = decoded.role === "admin" ? Admin : decoded.role === "agent" ? Agent : User;
    req.user = await Model.findById(decoded.id).select("-password");

    if (!req.user) return res.status(401).json({ message: "Account not found" });
    next();
  } catch (error) {
    res.status(401).json({ message: "Invalid or expired token" });
  }
};

export const authorize = (...roles) => (req, res, next) => {
  if (!roles.includes(req.user.role)) {
    return res.status(403).json({ message: "Access denied" });
  }
  return next();
};
```

backend/src/middleware/upload.js

```
js

import multer from "multer";
import path from "path";

const storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, "uploads/"),
  filename: (req, file, cb) => {
    const uniqueName = `${Date.now()}-${file.originalname}`;
    cb(null, uniqueName);
  }
});

const fileFilter = (req, file, cb) => {
  const allowed = /jpeg|jpg|png|pdf/;
  const isValid = allowed.test(path.extname(file.originalname).toLowerCase());
  cb(isValid ? null : new Error("Only images and PDFs are allowed"), isValid);
};

export default multer({ storage, fileFilter });
```

backend/src/routes/userRoutes.js

```
js

import express from "express";
import { bookShipment, loginUser, registerUser, shipmentHistory, trackShipment } from "../controllers/userController.js";
import { protect } from "../middleware/authMiddleware.js";

const router = express.Router();

router.post("/signup", registerUser);
router.post("/login", loginUser);
router.post("/shipments", protect, bookShipment);
router.get("/shipments/history", protect, shipmentHistory);
router.get("/track/:trackingId", trackShipment);

export default router;
```

backend/src/routes/adminRoutes.js

```
js

import express from "express";
import { adminLogin, adminSignup, assignAgent, getShipments, getUsers, updateDeliveryStatus } from "../controllers/adminController.js";
import { authorize, protect } from "../middleware/authMiddleware.js";

const router = express.Router();

router.post("/signup", adminSignup);
router.post("/login", adminLogin);
router.get("/users", protect, authorize("admin"), getUsers);
router.get("/shipments", protect, authorize("admin"), getShipments);
router.post("/assign-agent", protect, authorize("admin"), assignAgent);
router.patch("/shipments/:id/status", protect, authorize("admin"), updateDeliveryStatus);

export default router;
```

backend/src/routes/agentRoutes.js

```
js

import express from "express";
import { agentLogin, getAssignedShipments, updateAssignedShipmentStatus } from "../controllers/agentController.js";
import { authorize, protect } from "../middleware/authMiddleware.js";
import upload from "../middleware/upload.js";

const router = express.Router();

router.post("/login", agentLogin);
router.get("/assigned", protect, authorize("agent"), getAssignedShipments);
router.patch("/shipments/:id/status", protect, authorize("agent"), upload.single("proof"), updateAssignedShipmentStatus);

export default router;
```

Controllers

adminController.js → Handles admin-related operations such as managing users, shipments, and updating delivery status.

userController.js → Handles user operations such as registration, login, booking shipments, and tracking.

agentController.js → Handles delivery agent operations such as updating delivery status and managing assigned shipments.

db

config.js → Contains database connection setup using MongoDB and Mongoose.

middleware

authMiddleware.js → Middleware for authentication and authorization using JWT or session validation.

upload.js → Middleware for handling file uploads such as delivery proof images or documents.

models

Admin

Admins.js → Schema/model for admin accounts and credentials.

User

Users.js → Schema/model for user accounts and profile details.

Shipment

Shipments.js → Schema/model for shipment details including sender, receiver, and delivery status.

Tracking.js → Schema/model for tracking updates such as location, time, and shipment progress.

Delivery Agent

Agents.js → Schema/model for delivery agent details and assigned shipments.

routes

adminRoutes.js → Defines API routes for admin actions such as managing users, shipments, and assigning agents.

userRoutes.js → Defines API routes for user actions such as registration, login, booking shipments, and tracking.

agentRoutes.js → Defines API routes for delivery agent actions such as updating delivery status and viewing assigned deliveries.

FRONT-END DEVELOPMENT:

Frontend Reference Video:

<https://drive.google.com/file/d/15Fc-iHgybDu9ET0sc3BAQTAJQJUKj3er/view?usp=drivesdk>

Frontend Structure:

Frontend Folder Structure

```
txt

frontend/
  src/
    Admin/
      Ahome.jsx
      Alogin.jsx
      Asignup.jsx
      Anavbar.jsx
      Users.jsx
      Shipments.jsx
      Agents.jsx
      Orders.jsx
    User/
      Uhome.jsx
      Ulogin.jsx
      Usignup.jsx
      Unavbar.jsx
      BookShipment.jsx
      TrackShipment.jsx
      ShipmentHistory.jsx
      ShipmentItem.jsx
    Components/
      Navbar.jsx
      Footer.jsx
      Home.jsx
    ui/
      Button.jsx
      Input.jsx
```

frontend/src/Components/Navbar.jsx

```
jsx

import { Link } from "react-router-dom";

const Navbar = () => {
  return (
    <nav className="navbar">
      <Link to="/" className="brand">CourierTrack</Link>
      <div className="nav-links">
        <Link to="/track">Track</Link>
        <Link to="/login">User Login</Link>
        <Link to="/admin/login">Admin</Link>
      </div>
    </nav>
  );
};

export default Navbar;
```

frontend/src/Components/Footer.jsx

```
jsx

const Footer = () => {
  return (
    <footer className="footer">
      <p>CourierTrack - Fast, safe, and transparent courier delivery.</p>
    </footer>
  );
};

export default Footer;
```

frontend/src/Components/Home.jsx

```
jsx

import { Link } from "react-router-dom";
import Navbar from "../Navbar";
import Footer from "../Footer";

const Home = () => {
  return (
    <>
      <Navbar />
      <main className="home-page">
        <section className="hero">
          <h1>Track every courier from booking to doorstep.</h1>
          <p>Book shipments, monitor delivery progress, and manage courier operations in one system.</p>
          <div className="hero-actions">
            <Link to="/signup">Book Shipment</Link>
            <Link to="/track">Track Package</Link>
          </div>
        </section>
      </main>
      <Footer />
    </>
  );
};

export default Home;
```

frontend/src/Admin/Alogin.jsx

```
jsx

import { useState } from "react";

const Alogin = () => {
  const [form, setForm] = useState({ email: "", password: "" });

  const handleSubmit = async (event) => {
    event.preventDefault();
    const response = await fetch("/api/admin/login", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(form)
    });
    const data = await response.json();
    localStorage.setItem("adminToken", data.token);
  };

  return (
    <form onSubmit={handleSubmit} className="auth-card">
      <h2>Admin Login</h2>
      <input placeholder="Email" onChange={(e) => setForm({ ...form, email: e.target.value })} />
      <input type="password" placeholder="Password" onChange={(e) => setForm({ ...form, password: e.target.value })} />
      <button type="submit">Login</button>
    </form>
  );
};

export default Alogin;
```

frontend/src/Admin/Ahome.jsx

```
jsx

const Ahome = () => {
  return (
    <section className="dashboard">
      <h1>Admin Dashboard</h1>
      <div className="stats-grid">
        <article>Total Users: 120</article>
        <article>Total Shipments: 430</article>
        <article>Delivered: 310</article>
        <article>Pending: 45</article>
      </div>
    </section>
  );
};

export default Ahome;
```

frontend/src/Admin/Shipments.jsx

```
jsx

const Shipments = ({ shipments = [] }) => {
  return (
    <section>
      <h2>Manage Shipments</h2>
      <table>
        <thead>
          <tr>
            <th>Tracking ID</th>
            <th>Receiver</th>
            <th>Status</th>
            <th>Agent</th>
          </tr>
        </thead>
        <tbody>
          {shipments.map((shipment) => (
            <tr key={shipment._id}>
              <td>{shipment.trackingId}</td>
              <td>{shipment.receiver.name}</td>
              <td>{shipment.status}</td>
              <td>{shipment.agent?.name || "Not Assigned"}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </section>
  );
};

export default Shipments;
```

frontend/src/User/Ulogin.jsx

```
jsx

import { useState } from "react";

const Ulogin = () => {
  const [form, setForm] = useState({ email: "", password: "" });

  const handleSubmit = async (event) => {
    event.preventDefault();
    const response = await fetch("/api/users/login", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(form)
    });
    const data = await response.json();
    localStorage.setItem("userToken", data.token);
  };

  return (
    <form onSubmit={handleSubmit} className="auth-card">
      <h2>User Login</h2>
      <input placeholder="Email" onChange={(e) => setForm({ ...form, email: e.target.value })} />
      <input type="password" placeholder="Password" onChange={(e) => setForm({ ...form, password: e.target.value })} />
      <button type="submit">Login</button>
    </form>
  );
};

export default Ulogin;
```

frontend/src/User/BookShipment.jsx

```
jsx

import { useState } from "react";

const BookShipment = () => {
  const [form, setForm] = useState({ packageWeight: "", deliveryType: "Standard" });

  const bookShipment = async (event) => {
    event.preventDefault();
    await fetch("/api/users/shipments", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        Authorization: `Bearer ${localStorage.getItem("userToken")}`
      },
      body: JSON.stringify(form)
    });
  };

  return (
    <form onSubmit={bookShipment} className="shipment-form">
      <h2>Book New Shipment</h2>
      <input placeholder="Package Weight" onChange={(e) => setForm({ ...form, packageWeight: e.target.value })} />
      <select onChange={(e) => setForm({ ...form, deliveryType: e.target.value })}>
        <option>Standard</option>
        <option>Express</option>
        <option>Economy</option>
      </select>
      <button type="submit">Book Shipment</button>
    </form>
  );
};

export default BookShipment;
```

frontend/src/User/TrackShipment.jsx

```
jsx

import { useState } from "react";

const TrackShipment = () => {
  const [trackingId, setTrackingId] = useState("");
  const [result, setResult] = useState(null);

  const track = async (event) => {
    event.preventDefault();
    const response = await fetch(`/api/users/track/${trackingId}`);
    const data = await response.json();
    setResult(data);
  };

  return (
    <section className="track-page">
      <form onSubmit={track}>
        <h2>Track Shipment</h2>
        <input placeholder="Enter tracking ID" onChange={(e) => setTrackingId(e.target.value)} />
        <button type="submit">Track</button>
      </form>

      {result?.shipment && (
        <article className="tracking-card">
          <h3>{result.shipment.trackingId}</h3>
          <p>Status: {result.shipment.status}</p>
          <p>Receiver: {result.shipment.receiver.name}</p>
        </article>
      )}
    </section>
  );
};

export default TrackShipment;
```

frontend/src/User/ShipmentItem.jsx

```
jsx

const ShipmentItem = ({ shipment }) => {
  return (
    <article className="shipment-item">
      <h3>{shipment.trackingId}</h3>
      <p>Receiver: {shipment.receiver.name}</p>
      <p>Status: {shipment.status}</p>
      <p>Delivery Type: {shipment.deliveryType}</p>
    </article>
  );
};

export default ShipmentItem;
```

frontend/src/User/ShipmentHistory.jsx

```
jsx

import ShipmentItem from "../ShipmentItem";

const ShipmentHistory = ({ shipments = [] }) => {
  return (
    <section>
      <h2>Shipment History</h2>
      {shipments.length === 0 ? (
        <p>No shipments found.</p>
      ) : (
        shipments.map((shipment) => <ShipmentItem key={shipment._id} shipment={shipment} />)
      )}
    </section>
  );
};

export default ShipmentHistory;
```

Admin (src/Admin)

Ahome.jsx → Admin dashboard/homepage.

Alogin.jsx → Admin login page.

Asignup.jsx → Admin signup/registration page.

Anavbar.jsx → Navigation bar for admin panel.

Users.jsx → Admin panel to manage registered users.

Shipments.jsx → Admin view to manage all shipments.

Agents.jsx → Admin panel to manage delivery agents.

Orders.jsx → Admin view for monitoring delivery orders.

User (src/User)

Uhome.jsx → User homepage/dashboard.

Ulogin.jsx → User login page.

Usignup.jsx → User signup/registration page.

Unavbar.jsx → Navigation bar for user side.

BookShipment.jsx → UI for booking a new courier shipment.

TrackShipment.jsx → UI to track shipment using tracking ID.

ShipmentHistory.jsx → Displays all past and current shipments of the user.

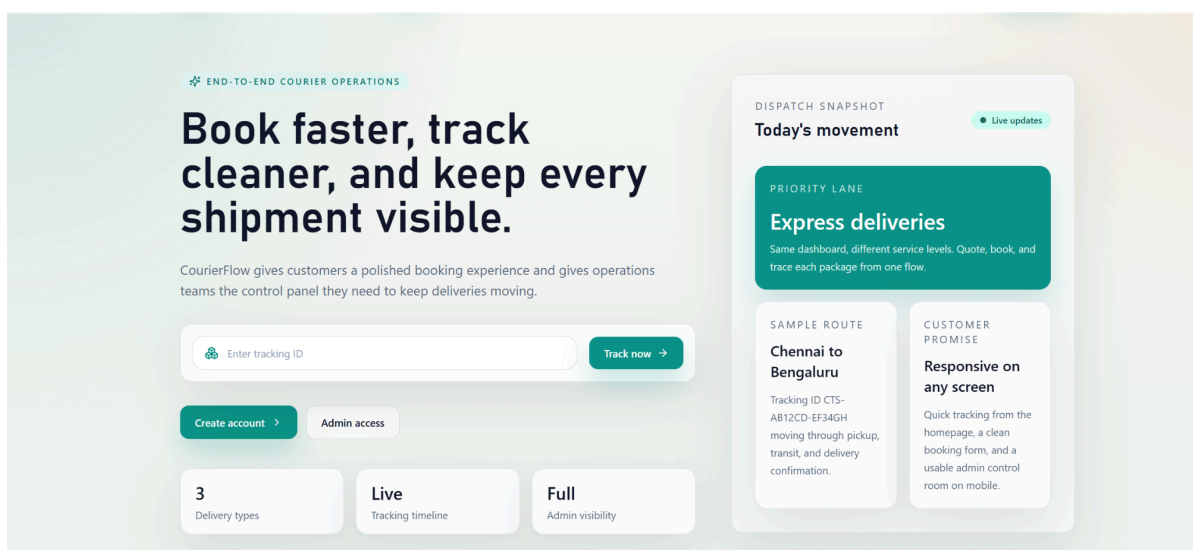
ShipmentItem.jsx → Component to display details of a single shipment.

Components (Reusable parts)

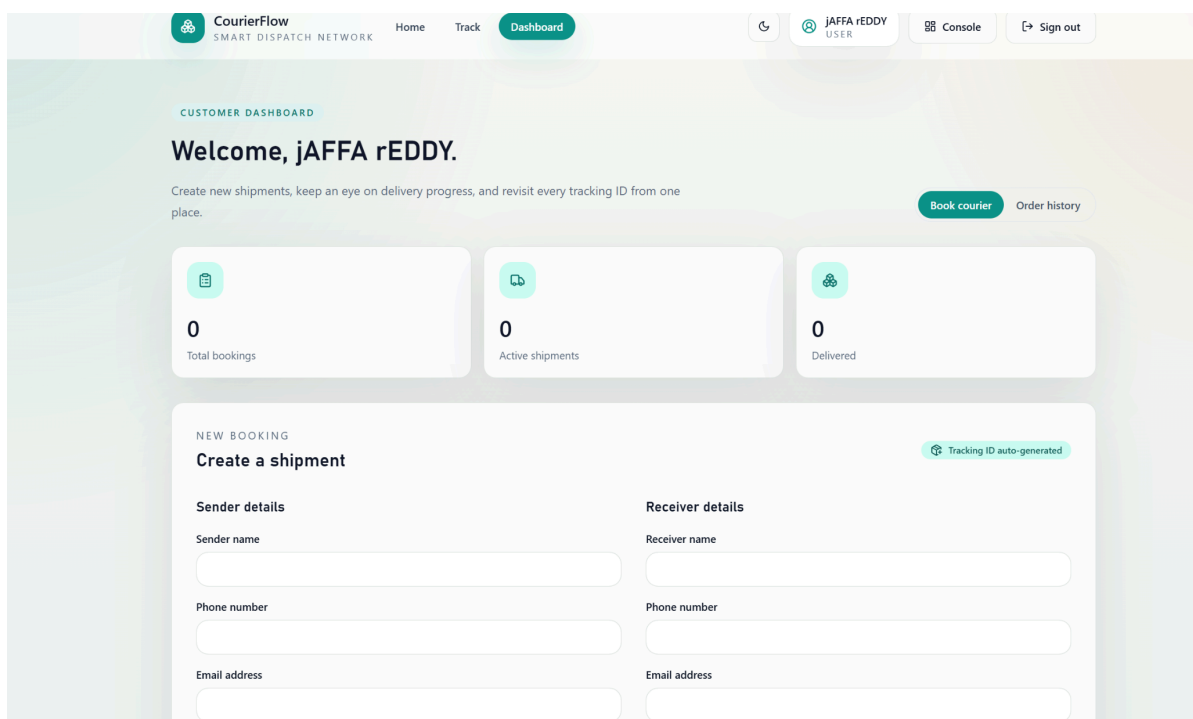
Navbar.jsx → Common navigation bar used across the application.
Footer.jsx → Footer component used across pages.
Home.jsx → Main landing page of the application.
ui/ → Contains shared UI components like buttons, forms, and modals.

OUTPUT SCREENSHOTS:

LANDING PAGE:



ADMIN PAGE:



DASHBOARD :

 **CourierFlow**
SMART DISPATCH NETWORK

HomeTrackDashboard

 jAFFA rEDDY
USER

Console

Sign out

CUSTOMER DASHBOARD

Welcome, jAFFA rEDDY.

Create new shipments, keep an eye on delivery progress, and revisit every tracking ID from one place.

Book courierOrder history


0
Total bookings


0
Active shipments


0
Delivered

NEW BOOKING

Create a shipment

Tracking ID auto-generated

Sender details

Sender name

Phone number

Email address

Receiver details

Receiver name

Phone number

Email address

CREATE A SHIPMENT :

 **CourierFlow**
SMART DISPATCH NETWORK

HomeTrackDashboard

 jAFFA rEDDY
USER

Console

Sign out

NEW BOOKING

Create a shipment

Tracking ID auto-generated

Sender details

Sender name

Phone number

Email address

Pickup address

Package weight (kg)

Receiver details

Receiver name

Phone number

Email address

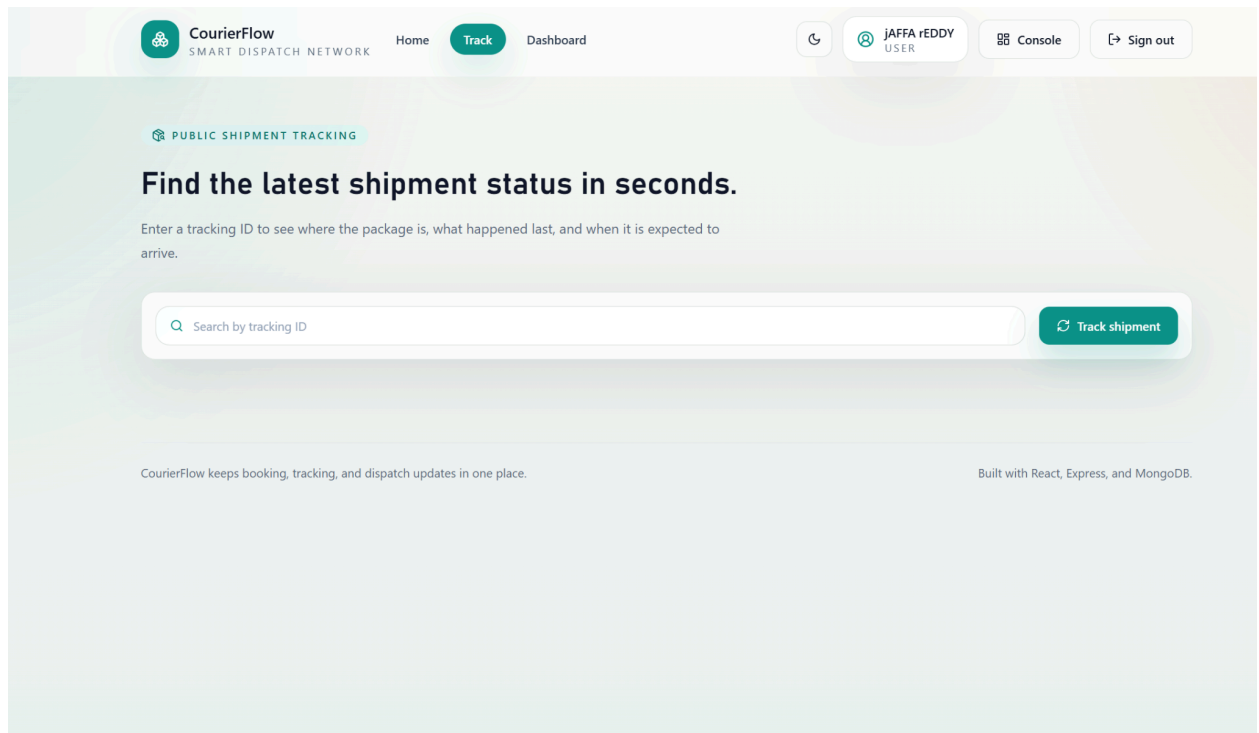
Delivery address

Delivery type

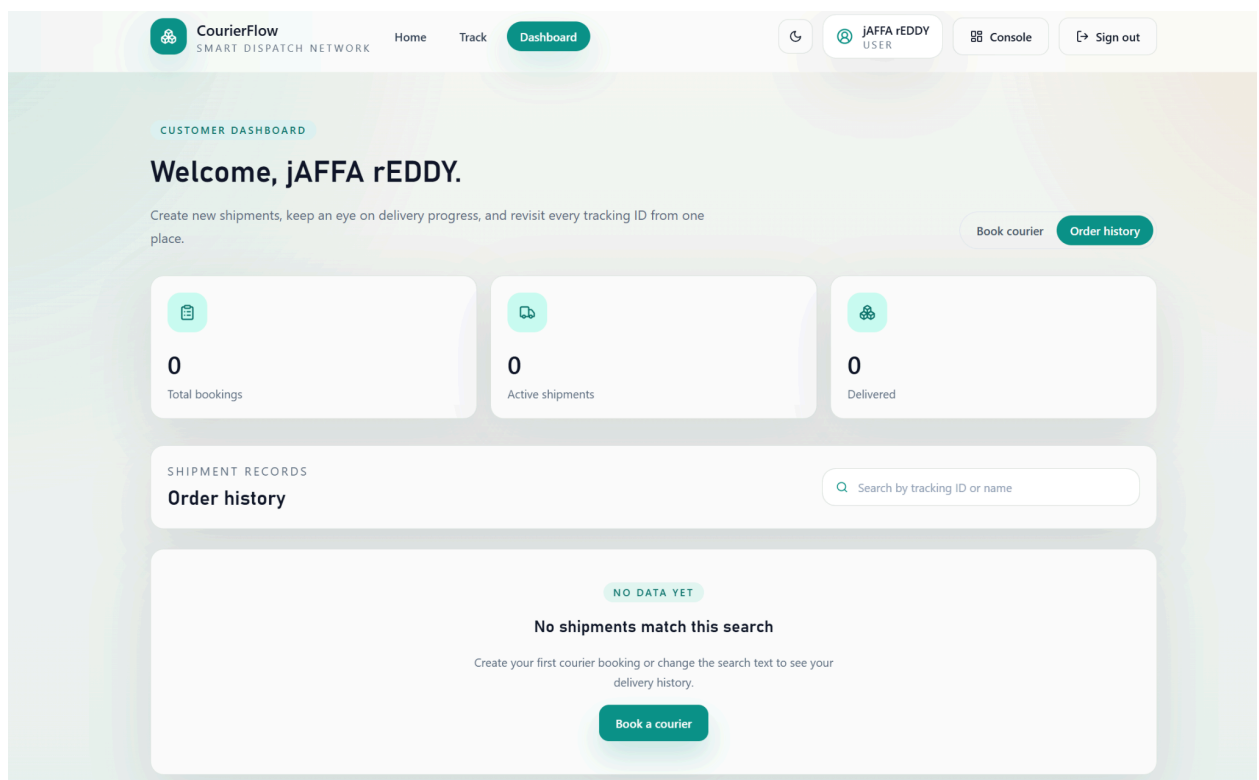
Create shipment

Reset form

TRACKING PAGE :



ORDER HISTORY :



SIGN UP :

 **CourierFlow**
SMART DISPATCH NETWORK

HomeTrack

  Track shipment  Sign in

CUSTOMER ACCESS

Track deliveries without losing the human feel.

Sign in to manage bookings, check recent activity, and keep each shipment close at hand.

→

Create bookings with automatic tracking IDs.

→

Review order history and delivery progress.

→

Jump from customer view to tracking results in one click.

ACCOUNT

Create your account

Sign inSign up

Full name

 John Doe

Email address

 name@example.com

Password

 Minimum 8 characters

Create account →

Admin team? Use the separate admin access page.[Open admin login](#)

SIGN IN:

 **CourierFlow**
SMART DISPATCH NETWORK

HomeTrack

  Track shipment  Sign in

CUSTOMER ACCESS

Track deliveries without losing the human feel.

Sign in to manage bookings, check recent activity, and keep each shipment close at hand.

→

Create bookings with automatic tracking IDs.

→

Review order history and delivery progress.

→

Jump from customer view to tracking results in one click.

ACCOUNT

Welcome back

Sign inSign up

Email address

 name@example.com

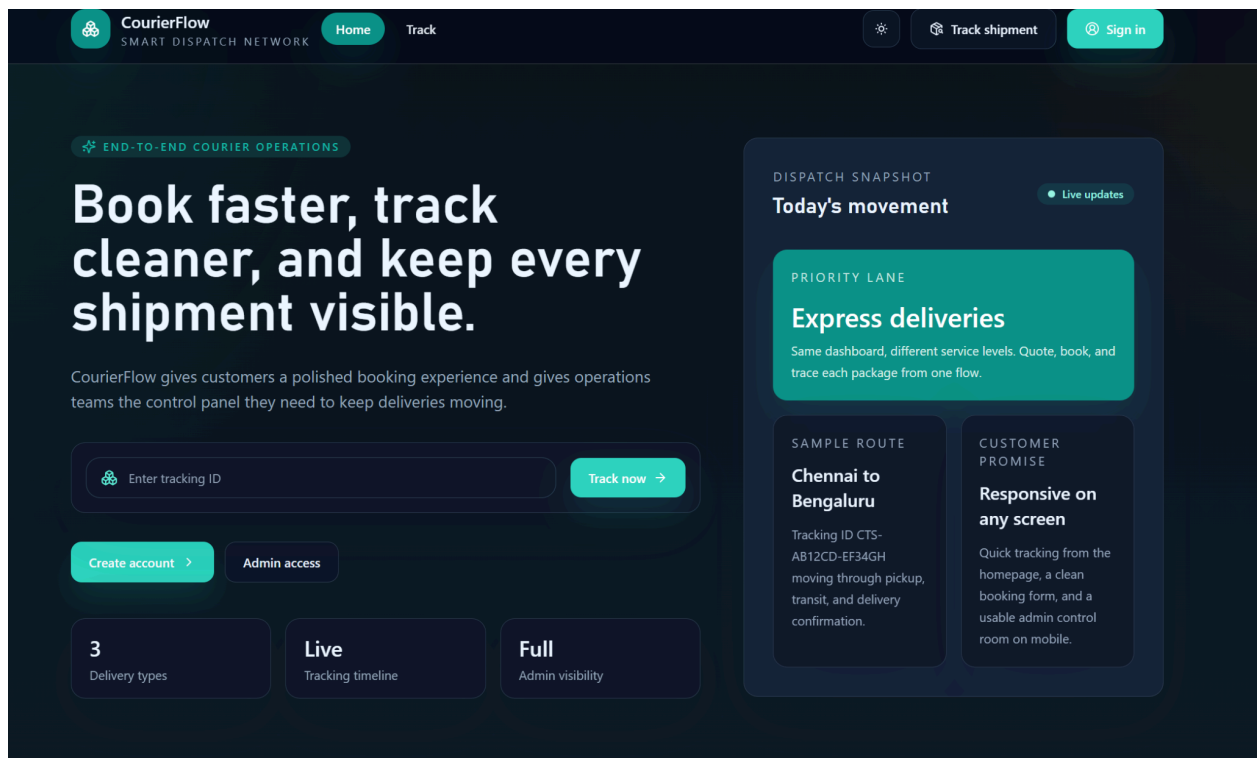
Password

 Minimum 8 characters

Sign in →

Admin team? Use the separate admin access page.[Open admin login](#)

DARK MODE :



Demo Video Link:

<https://drive.google.com/file/d/14NAFtKnxzjQeNEF0oSuW3r3EMMYc-c-i7/view?usp=drivesdk>

Code Drive Link:

<https://drive.google.com/drive/folders/1g2erE97ihlaS3HLIS4Mx3GYIn-i7QpRBZ>